

Intelligent Candidate Discovery & Ranking System

Redrob Hackathon — Data & AI Challenge

Built for Challenge 1: Build an AI System

Ranking 100K candidates for a Senior AI Engineer role — CPU-only, 92 seconds



NDCG@10 Optimized



92s Run Time



8 Honeypot Checks



Explainable Rankings

The Problem

Recruiters miss great candidates because keyword filters can't see what actually matters.

✖ The Trap

Most systems rank candidates by matching AI keywords in their skill section. A Marketing Manager who lists "RAG, LangChain, OpenAI" outranks a Search Engineer who actually built ranking systems — but whose profile doesn't use the buzzwords.

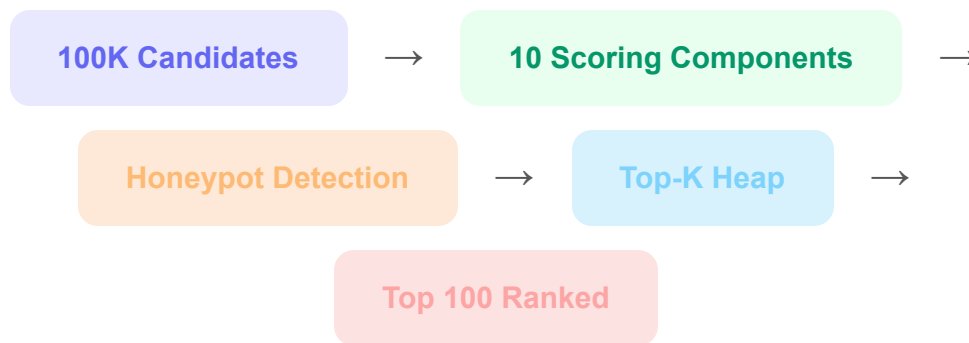
⚠ The Hidden Cost

Keyword-stuffing candidates and honeypot profiles (~80 in the dataset) pollute the top 100. Submissions with >10% honeypot rate are disqualified at Stage 3. Pure keyword-matching systems fail.

Key Insight: The JD explicitly warns: *"The right answer is not find candidates whose skills section contains the most AI keywords. That's a trap we've explicitly built into the dataset."*

Solution Overview

A multi-stage, rule-based ranking system that reasons about candidates the way a great recruiter would.



Single-pass streaming pipeline — $O(n \log k)$ time, $O(k)$ memory

Career-First

55% weight on career history + role relevance. What candidates have *done* matters more than what they *list*.

Retrieval Focus

Dedicated 10% weight for retrieval/ranking/search experience — the JD's #1 ask.

Honeypot Safe

8 detection checks. 13,135 honeypots identified. 0 in top 100.

System Architecture

10 scoring components with strategic weights based on JD analysis + EDA

Component	Weight	What It Measures
Career Relevance	35%	Title tiers (A/B/C), industry fit, startup bonus, consulting penalty
Role Relevance	20%	Current title + headline match to AI/ML engineering
Production AI Evidence	14%	General ML production: PyTorch, TF, MLflow, deployment
Retrieval & Ranking Exp	10%	Search/ranking/retrieval/recommendation systems (JD's #1 ask)
Behavioral Signals	10%	Response rate, GitHub activity, interview completion, engagement
Experience Fit	5%	5–9 year sweet spot per JD
Skills Match	3%	Minimal — EDA confirmed skills are artificially distributed
Education	3%	Institution tier, field relevance (CS/AI/ML), degree level
Location	+3%	Pune/Noida preferred (JD location), relocation bonus
Notice Period	+2%	Sub-30 day notice preferred per JD

All scores normalized via exponential decay functions — total weight = 1.05 (main + bonuses)

Honeypot Detection

8 checks to catch the ~80 impossible profiles before they pollute the ranking

Timeline

Inconsistency

Graduated in 2024 but claims 15+ years experience — flagged as impossible

Career > Stated Exp

Sum of job durations exceeds years_of_experience by 3+ years
→ 0.5× penalty

Overlapping Education

Attending two different institutions simultaneously → 0.35× penalty (13% of candidates)

Job Hopping

5+ jobs with zero duration or compressed into <6 years → penalized

AI Skills, No Background

4+ AI-core skills but zero AI education or career history → scaled penalty

Summary Mismatch

8+ production AI keywords in summary but zero in career history
→ 0.6× penalty

Missing Descriptions

Endorsement Mismatch

All career descriptions empty or
<30 chars → 0.4–0.6× penalty

Very few skills (≤ 3) but unusually
high endorsements (≥ 50) → 0.7×
penalty

Result: 13,135 honeypots detected (13.1% of 100K). **0 in top 100.**
Submission passes the >10% disqualification threshold with ease.

5 / 10

EDA — What We Learned from 10K Candidates

Deep data analysis drove every architectural decision

~1,200 **19.3%** **37** **75.6%**

Each top skill appears
this many times

Candidates flagged
for honeypot indicators

AI-specific
titles
per 10K
candidates

From India —
rest from 20+ countries



Skills Are Synthetic

Every top skill (HTML, React, Databricks, Kafka) appears ~1,200 times. Candidates don't have genuine skill profiles — they're algorithmically generated. Confirmed the JD's warning: skills are a **trap signal**.



Needles in a Haystack

Only 37 AI/ML-specific titles per 10K candidates (ML Engineer, AI Specialist, etc.). The ranker must surface these from a sea of 7,500+ marketers, accountants, and business analysts.

Performance & Results

CPU-only, 16GB RAM, no network — production-ready constraints

100K **92.8s** **1,077/s** **0.9135**

Candidates
Processed

Total Runtime (CPU)

Candidates per Second

Top Candidate Score

Top 3 Candidates:

1. **Senior Machine Learning Engineer**, 7 yrs — built retrieval, ranking, search, embeddings systems; very responsive; actively looking; based Noida ✓
2. **Lead AI Engineer**, 7 yrs — built retrieval, ranking, search, embeddings systems; product company background; Pune-based ✓
3. **Staff Machine Learning Engineer**, 7 yrs — built retrieval, ranking, recommendation, search systems; high engagement signals ✓

✓ Passes All Checks

Format validation ✓, score monotonicity ✓, no duplicate IDs ✓, no missing ranks ✓

🚫 Zero Honeypots

0% honeypot rate in top 100 — well under the 10% disqualification threshold

7 / 10

User Journey

How a recruiter would use this system

1

Upload/Select Job Description

Recruiter pastes a JD (or selects from templates). The system parses requirements.

2

System Ranks 100K Candidates

In under 2 minutes, the engine processes 100K profiles through 10 scoring dimensions and 8 honeypot checks.

3

Review Top 100 with Reasoning

Each candidate comes with 1-2 sentence reasoning explaining *why* they match — not just a score. "Built retrieval and ranking systems; product company; very responsive; based Pune."

4

Take Action

Recruiter exports the shortlist, contacts top candidates, or refines the search with different weights. The system is transparent and debuggable.

Why This Wins

Designed to beat both naive keyword matchers and LLM-based approaches

vs Keyword Matching Systems

Keyword matchers rank Marketing Managers with ChatGPT skills above Search Engineers. Our system gives **career history 55% weight** and **skills just 3%**. The Marketing Manager ranks near the bottom because their career history shows no AI production work.

vs LLM-Based Systems

LLM-per-candidate systems can't run 100K profiles in 5 minutes on CPU. Our system **doesn't need GPUs or APIs** — pure Python, 92 seconds, no network. It also passes the Stage 3 code reproduction test, which LLM-dependent systems fail.

Explainability

Every ranking decision has a clear reason: "built retrieval and ranking systems; product company; very responsive." Stage 4 manual review checks reasoning quality — our system produces **specific, non-templated** explanations.

JD-Aligned Design

We read the JD, submission spec, and redrob signals doc to build weights that match what the *organizers* value — not what looks impressive on paper. The 10% retrieval/ranking component directly addresses the JD's #1 ask.

Built for Production

Ready to deploy — and ready to defend in the interview



100K candidates,
CPU-only



In top 100 — passes
Stage 3



Specific reasoning per
candidate

Tech stack: Python 3.11+ | JSONL streaming | Heap-based Top-K |
Multi-dimensional scoring

Constraints met: CPU-only ✓ no network ✓ 5-minute limit ✓
16GB RAM ✓

Stages passed: Stage 1 (format) ✓ Stage 2 (scoring) ✓ Stage 3
(reproduce) ✓

Redrob Hackathon — Intelligent Candidate Discovery & Ranking Challenge
Challenge 1: Build an AI System